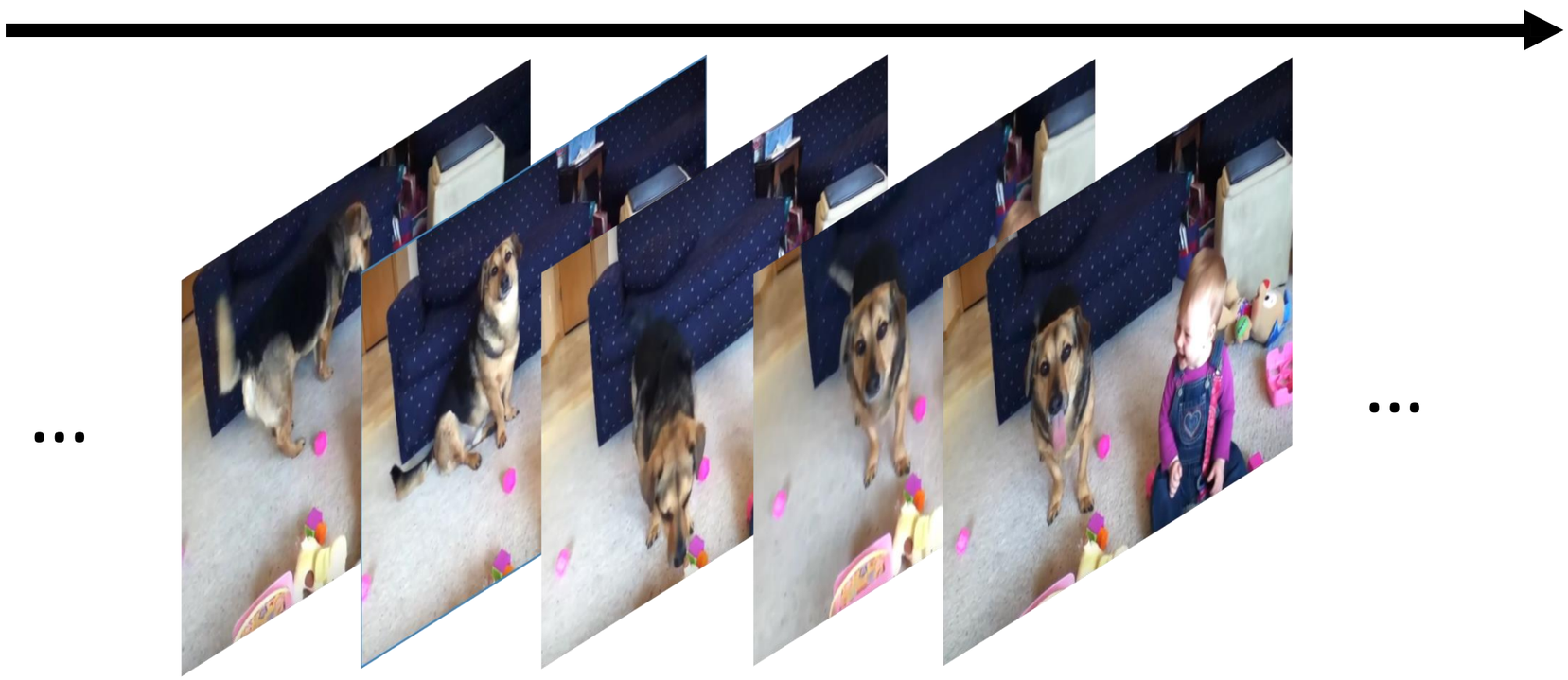
Video Understanding

Video = 2D + Time

A video is a sequence of images

4D tensor: T x 3 x H x W

(or 3 x T x H x W)



Example task: Video Classification

Input video:
T x 3 x H x W

Swimming
Running
Jumping
Eating
Standing

Problem: Videos are big!

Input video:
T x 3 x H x W

Videos are ~30 frames per second (fps)

Size of uncompressed video
(3 bytes per pixel):

SD (640 x 480): ~1.5 GB per minute
HD (1920 x 1080): ~10 GB per minute

Solution: Train on short clips: low fps and low spatial resolution
e.g. T = 16, H=W=112
(3.2 seconds at 5 fps, 588 KB)

Training on Clips

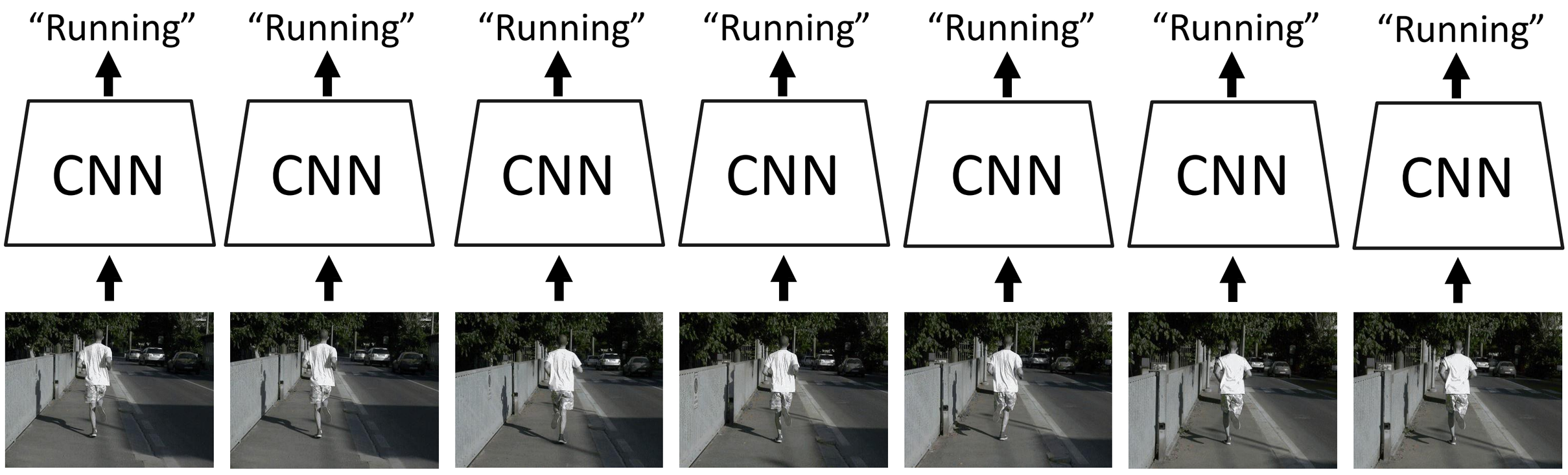
Raw video: Long, high FPS

Training: Train model to classify short clips with low FPS

Testing: Run model on different clips, average predictions

Video Classification: Single-Frame CNN

Simple idea: train normal 2D CNN to classify video frames independently!
(Average predicted probs at test-time)
Often a very strong baseline for video classification

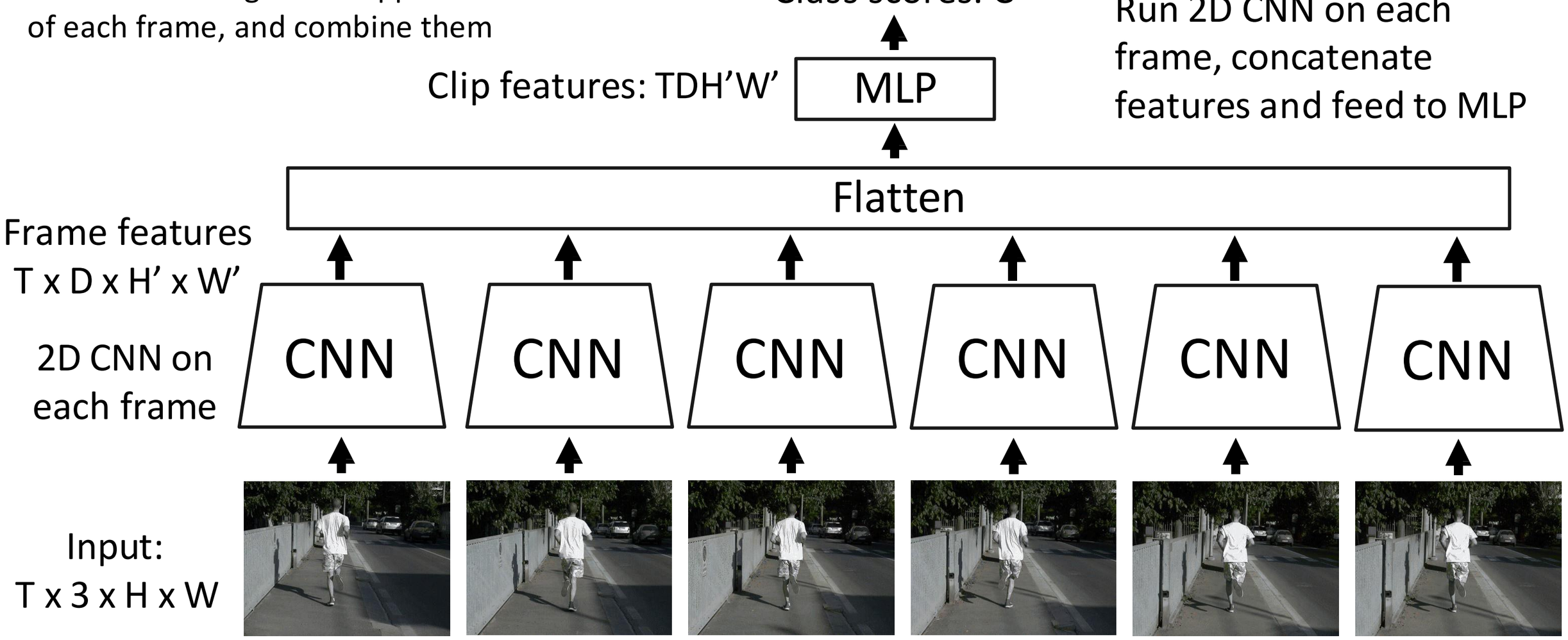


Video Classification: Late Fusion (with FC layers)

Intuition: Get high-level appearance of each frame, and combine them

Class scores: C

Run 2D CNN on each frame, concatenate features and feed to MLP

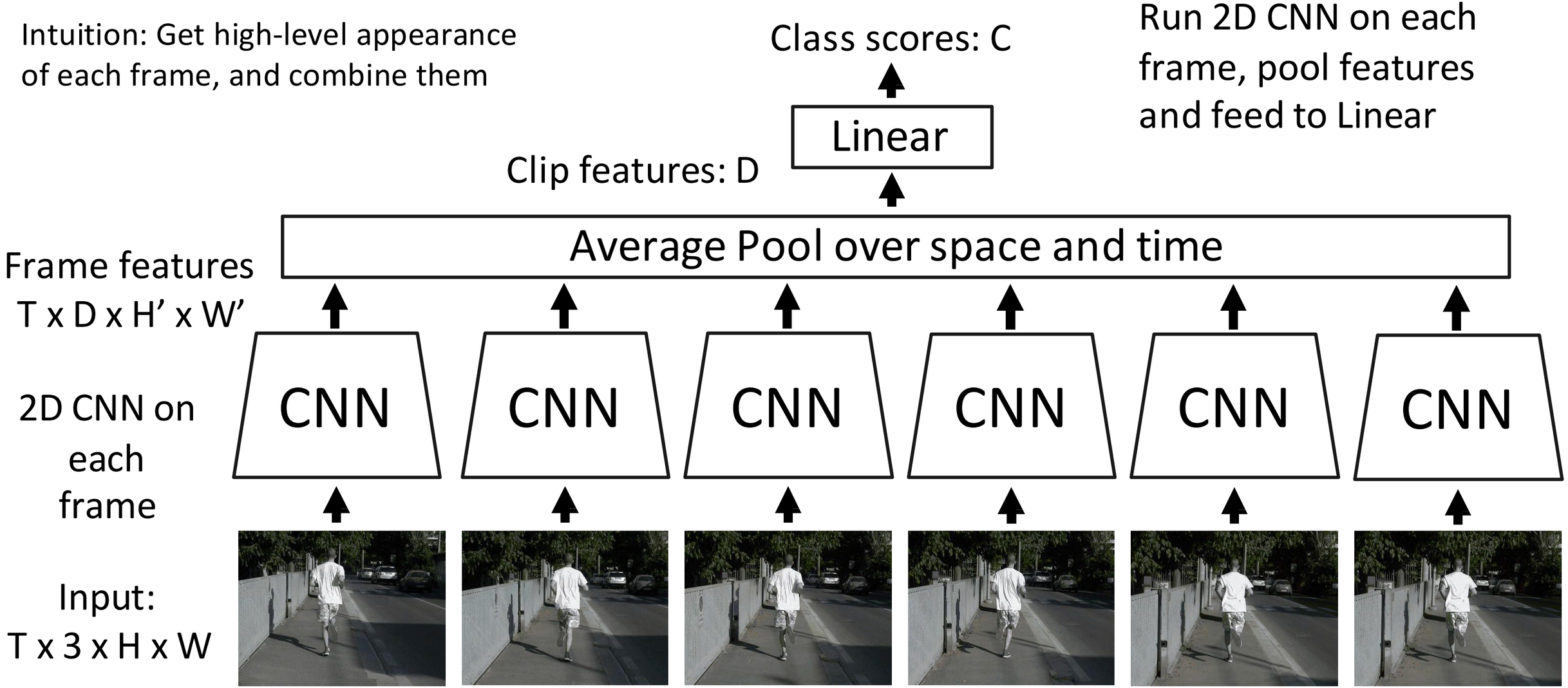


Karpathy et al, "Large-scale Video Classification with Convolutional Neural Networks", CVPR 2014

Video Classification: Late Fusion (with pooling)

Intuition: Get high-level appearance of each frame, and combine them

Run 2D CNN on each frame, pool features and feed to Linear



Video Classification: Early Fusion

Intuition: Compare frames with very first conv layer, after that normal 2D CNN

Class scores: C

Rest of the network is standard 2D CNN

First 2D convolution collapses all temporal information:
Input: 3T x H x W
Output: D x H x W

2D CNN

Reshape:
3T x H x W

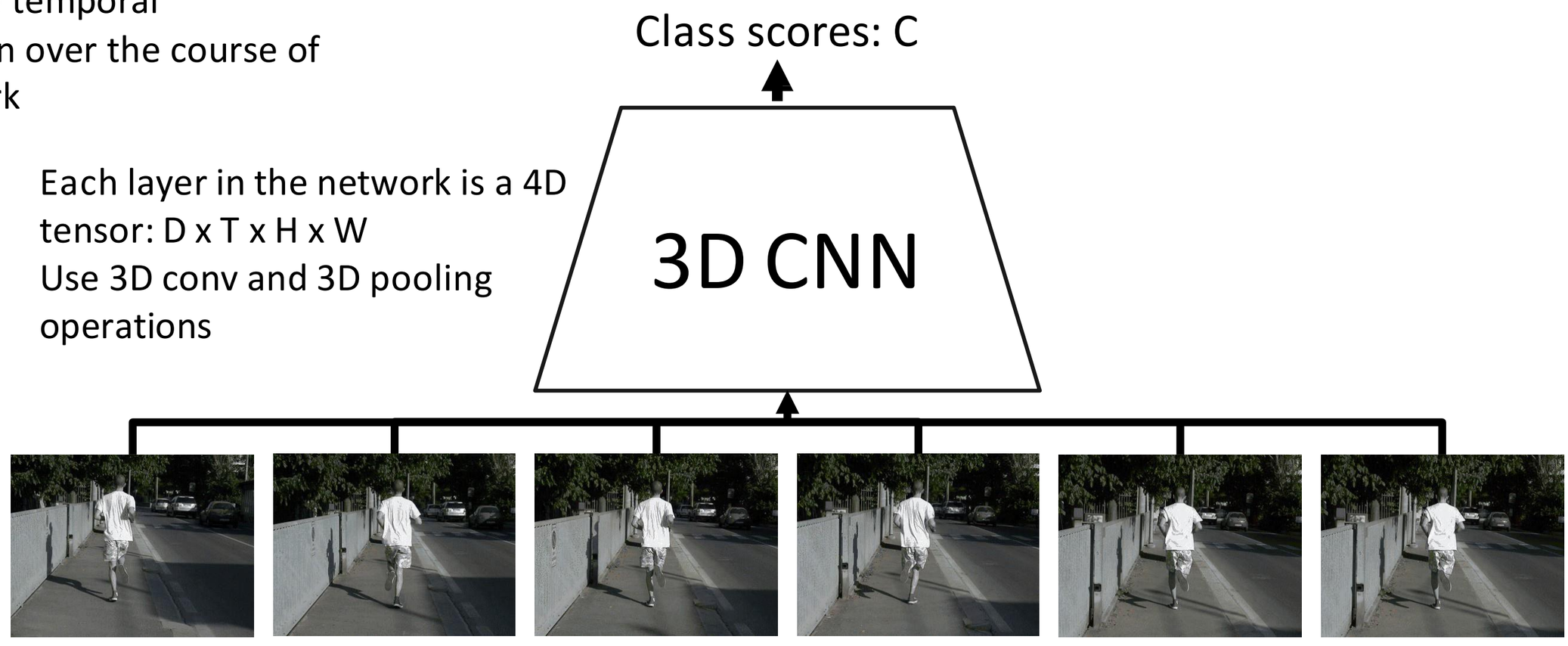
Input:
T x 3 x H x W

Karpathy et al, “Large-scale Video Classification with Convolutional Neural Networks”, CVPR 2014

Video Classification: 3D CNN

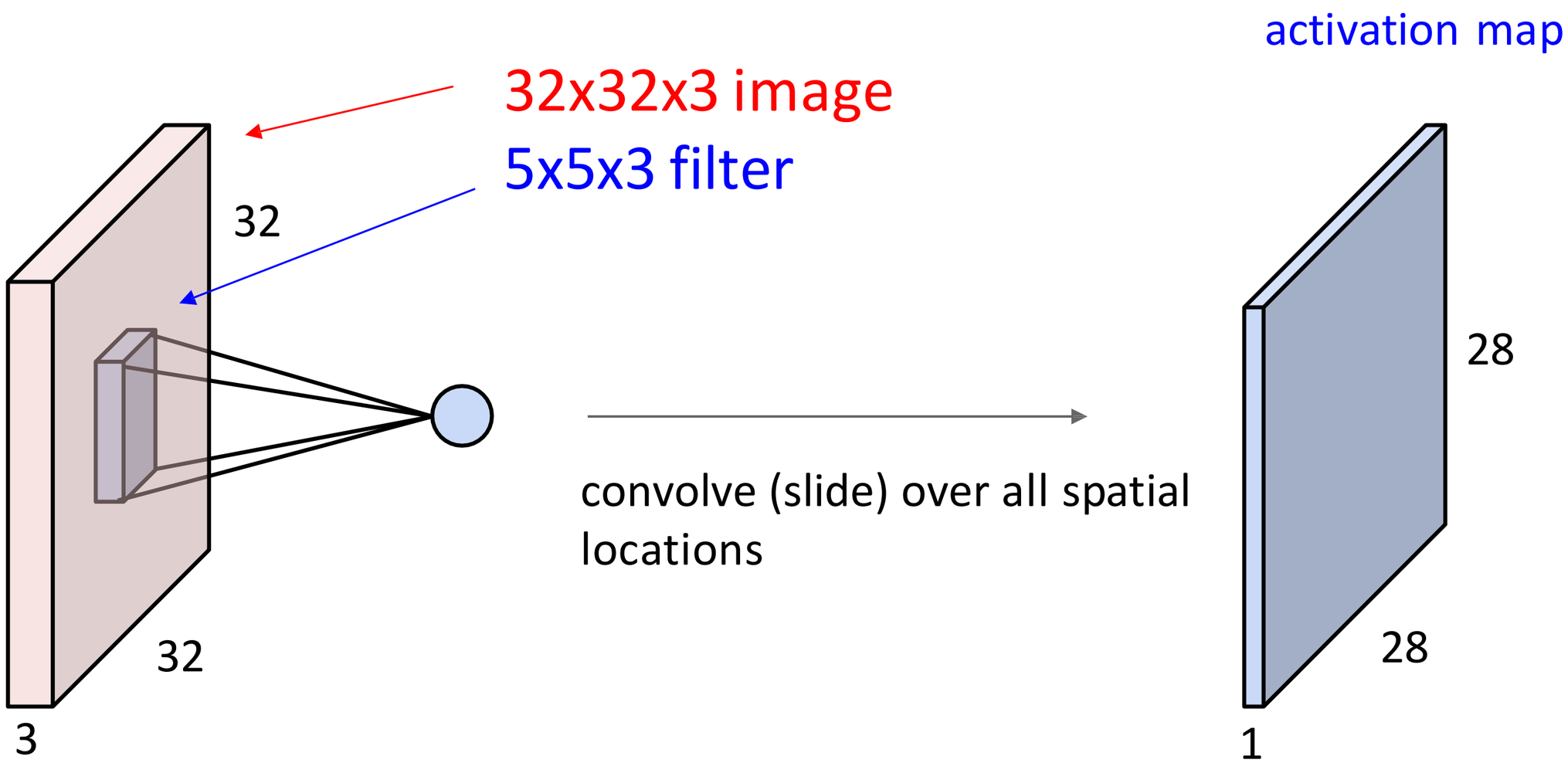
Intuition: Use 3D versions of convolution and pooling to slowly fuse temporal information over the course of the network

Each layer in the network is a 4D tensor: D x T x H x W
Use 3D conv and 3D pooling operations

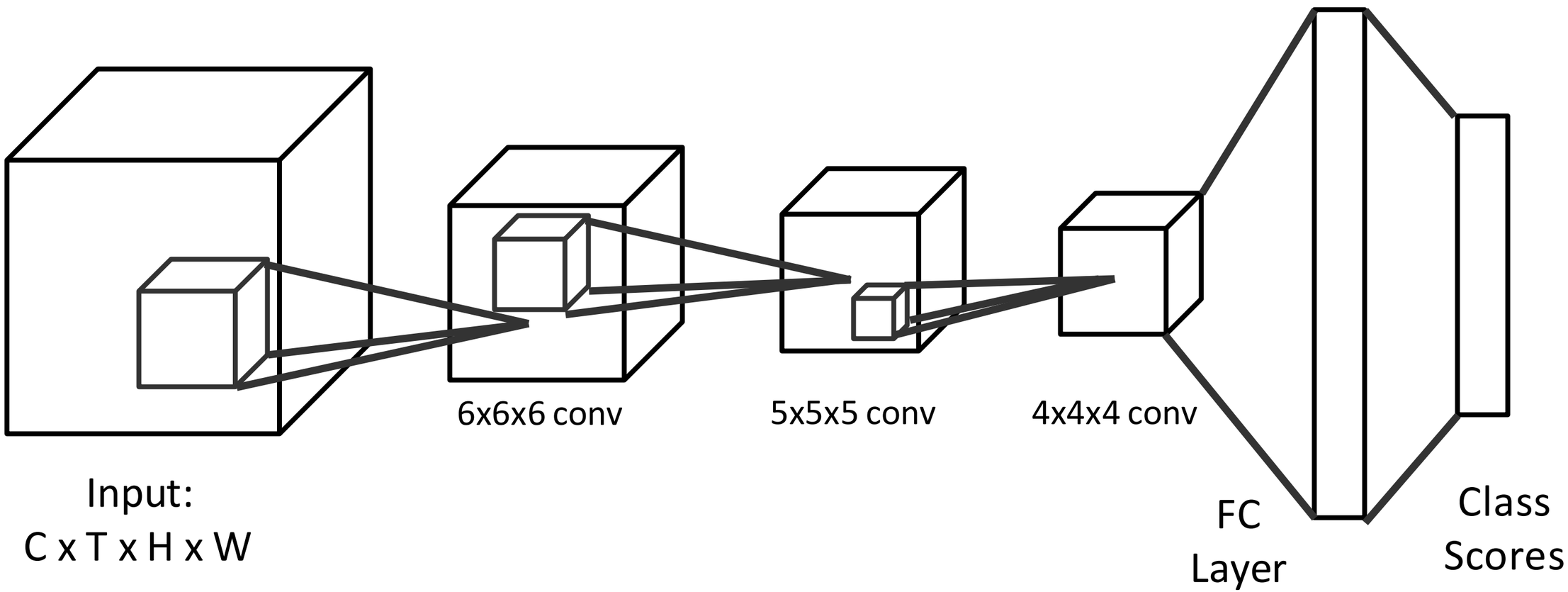


Ji et al, “3D Convolutional Neural Networks for Human Action Recognition”, TPAMI 2010 ; Karpathy et al, “Large-scale Video Classification with Convolutional Neural Networks”, CVPR 2014

Convolution Layer



3D Convolution



Early Fusion vs Late Fusion vs 3D CNN

What is the difference?

	Layer	Size **(C x T x H x W)**	Receptive Field **(T x H x W)**	
Late Fusion	Input	3 x 20 x 64 x 64		
	Conv2D(3x3, 3->12)	12 x 20 x 64 x 64	1 x 3 x 3	Build slowly in space, All-at-once in time at end
	Pool2D(4x4)	12 x 20 x 16 x 16	1 x 6 x 6	
	Conv2D(3x3, 12->24)	24 x 20 x 16 x 16	1 x 14 x 14	
	GlobalAvgPool	24 x 1 x 1 x 1	20 x 64 x 64	
Early Fusion	Input	3 x 20 x 64 x 64		
	Conv2D(3x3, 3*20->12)	12 x 64 x 64	20 x 3 x 3	Build slowly in space, All-at-once in time at start
	Pool2D(4x4)	12 x 16 x 16	20 x 6 x 6	
	Conv2D(3x3, 12->24)	24 x 16 x 16	20 x 14 x 14	
	GlobalAvgPool	24 x 1 x 1	20 x 64 x 64	
3D CNN	Input	3 x 20 x 64 x 64		
	Conv3D(3x3x3, 3->12)	12 x 20 x 64 x 64	3 x 3 x 3	Build slowly in space, Build slowly in time "Slow Fusion"
	Pool3D(4x4x4)	12 x 5 x 16 x 16	6 x 6 x 6	
	Conv3D(3x3x3, 12->24)	24 x 5 x 16 x 16	14 x 14 x 14	
	GlobalAvgPool	24 x 1 x 1	20 x 64 x 64	

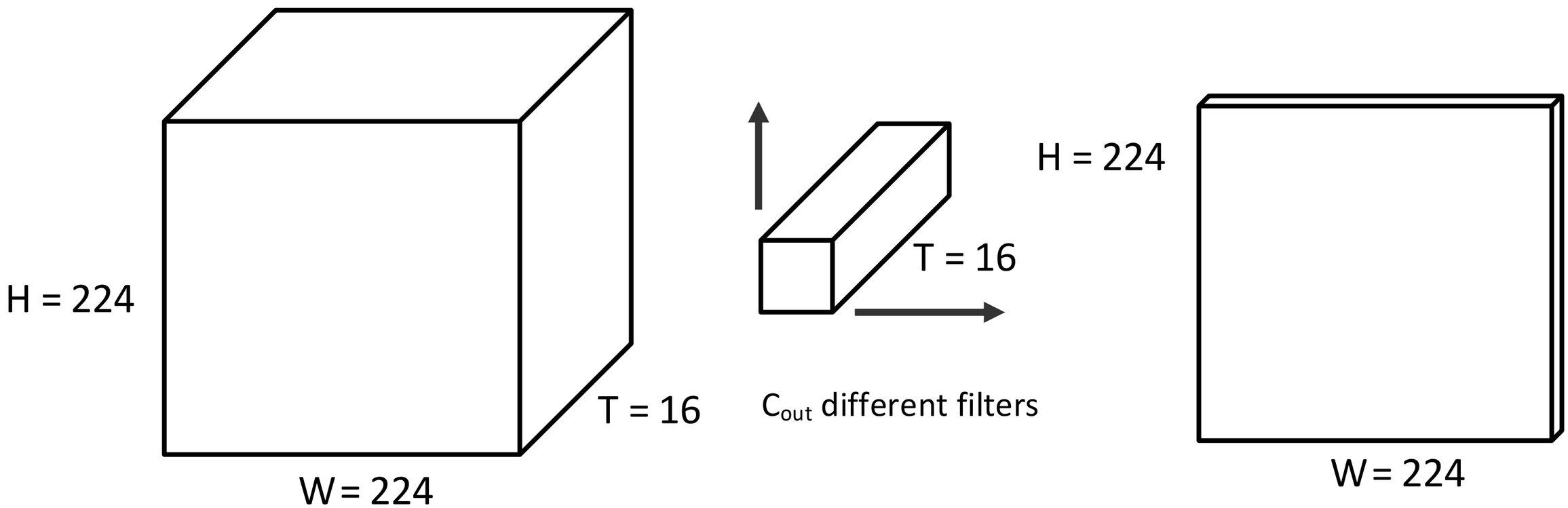
(Small example architectures, in practice much bigger)

2D Conv (Early Fusion) vs 3D Conv (3D CNN)

Input: C_{in} x T x H x W
(3D grid with C_{in}-dim feat at each point)

Weight:
C_{out} x C_{in} x T x 3 x 3
Slide over x and y

Output:
C_{out} x H x W
2D grid with C_{out} – dim feat at each point

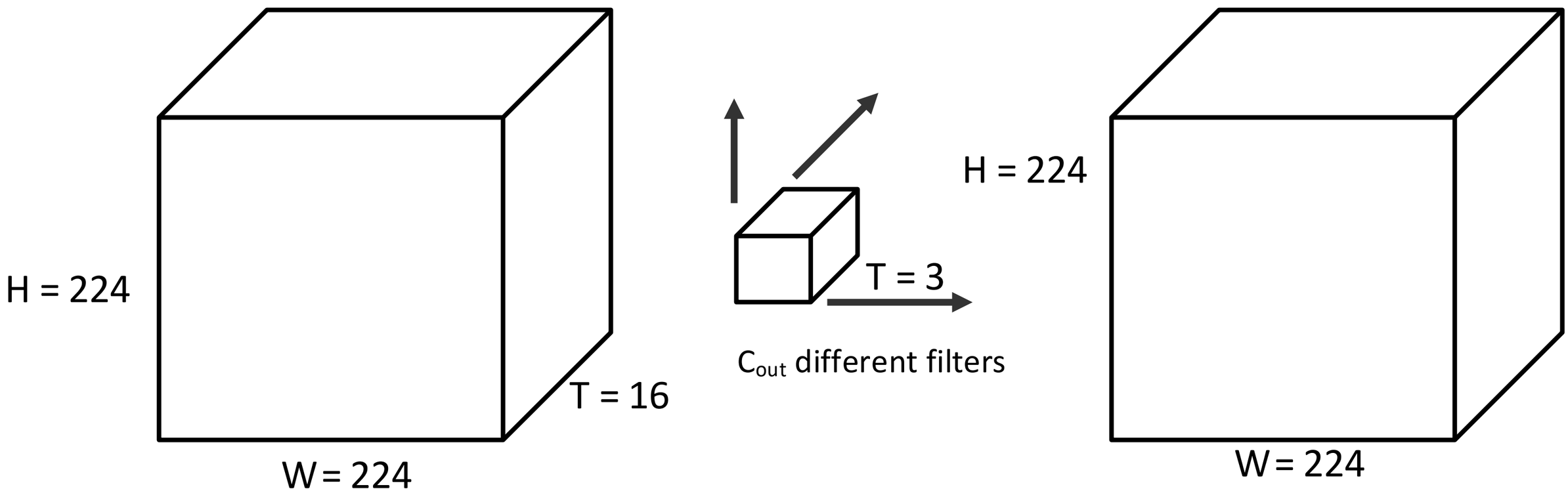


2D Conv (Early Fusion) vs 3D Conv (3D CNN)

Input: C_{in} x T x H x W
(3D grid with C_{in}-dim feat at each point)

Weight:
C_{out} x C_{in} x 3 x 3 x 3
Slide over x and y

Output:
C_{out} x T x H x W
3D grid with C_{out}–dim feat at each point



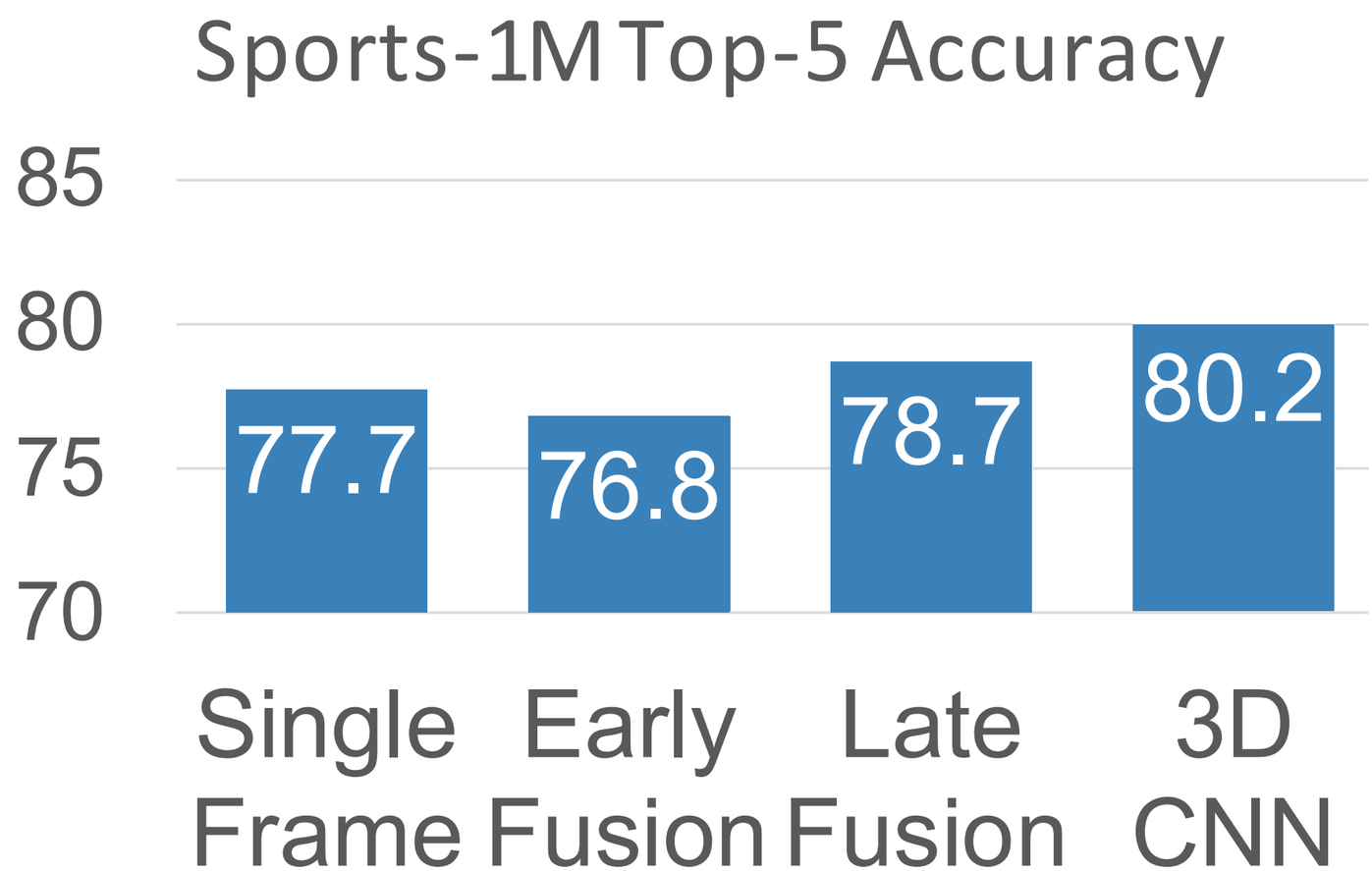
Example Video Dataset: Sports-1M

1 million YouTube videos annotated with labels for 487 different types of sports

Ground Truth
Correct prediction
Incorrect prediction

Karpathy et al, "Large-scale Video Classification with Convolutional Neural Networks", CVPR 2014

Early Fusion vs Late Fusion vs 3D CNN



Single Frame model works well – always try this first!

3D CNNs have improved a lot since 2014!

Karpathy et al, "Large-scale Video Classification with Convolutional Neural Networks", CVPR 2014

C3D: The VGG of 3D CNNs

3D CNN that uses all 3x3x3 conv and 2x2x2 pooling
(except Pool1 which is 1x2x2)

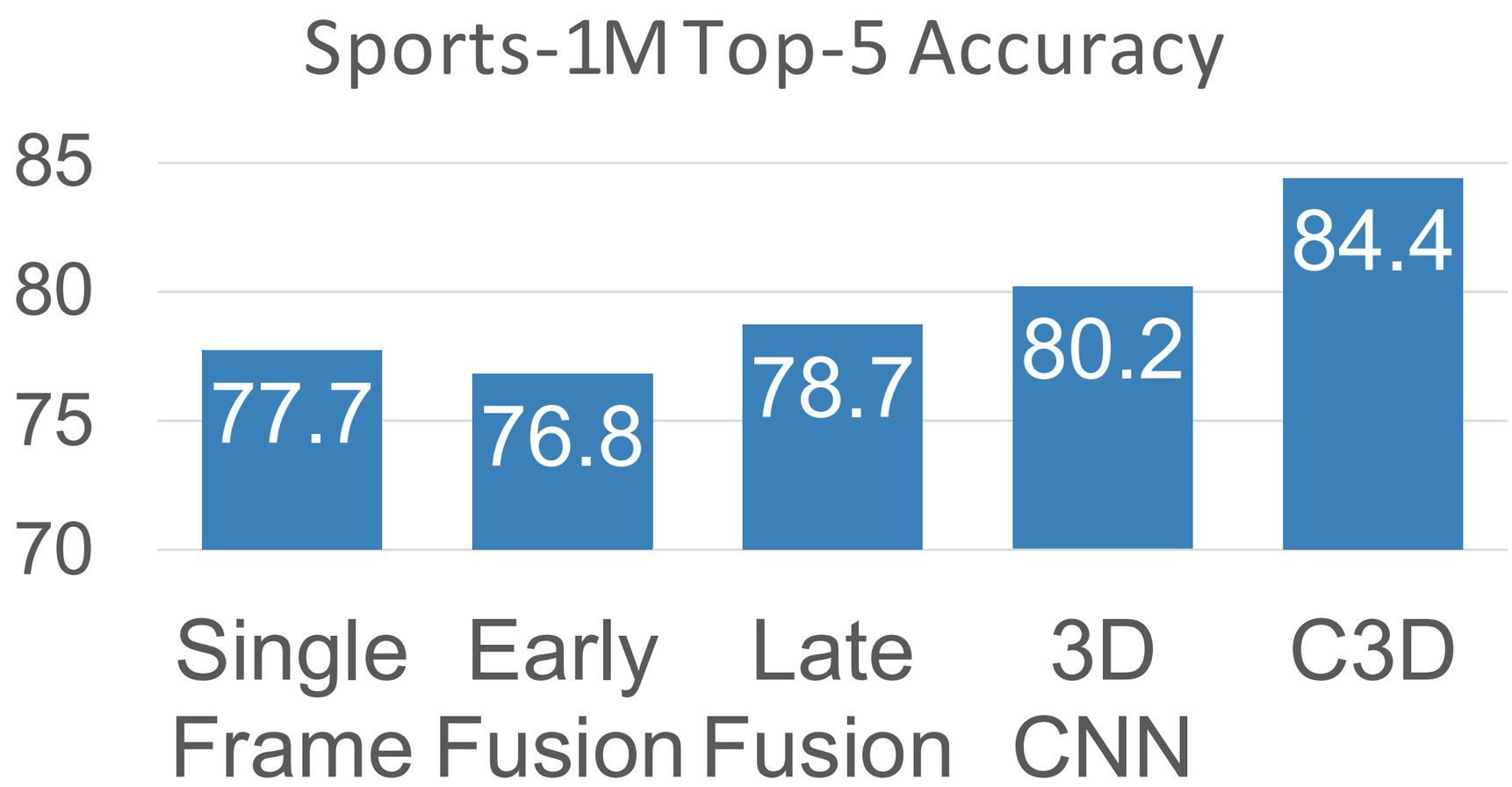
Released model pretrained on Sports-1M: Many people used this as a video feature extractor

Problem: 3x3x3 conv is very expensive!
AlexNet: 0.7 GFLOP
VGG-16: 13.6 GFLOP
C3D: 39.5 GFLOP (2.9x VGG!)

Layer	Size	MFLOPs
Input	3 x 16 x 112 x 112	
Conv1 (3x3x3)	64 x 16 x 112 x 112	1.04
Pool1 (1x2x2)	64 x 16 x 56 x 56	
Conv2 (3x3x3)	128 x 16 x 56 x 56	11.10
Pool2 (2x2x2)	128 x 8 x 28 x 28	
Conv3a (3x3x3)	256 x 8 x 28 x 28	5.55
Conv3b (3x3x3)	256 x 8 x 28 x 28	11.10
Pool3 (2x2x2)	256 x 4 x 14 x 14	
Conv4a (3x3x3)	512 x 4 x 14 x 14	2.77
Conv4b (3x3x3)	512 x 4 x 14 x 14	5.55
Pool4 (2x2x2)	512 x 2 x 7 x 7	
Conv5a (3x3x3)	512 x 2 x 7 x 7	0.69
Conv5b (3x3x3)	512 x 2 x 7 x 7	0.69
Pool5	512 x 1 x 3 x 3	
FC6	4096	0.51
FC7	4096	0.45
FC8	C	0.05

Tran et al, "Learning Spatiotemporal Features with 3D Convolutional Networks", ICCV 2015

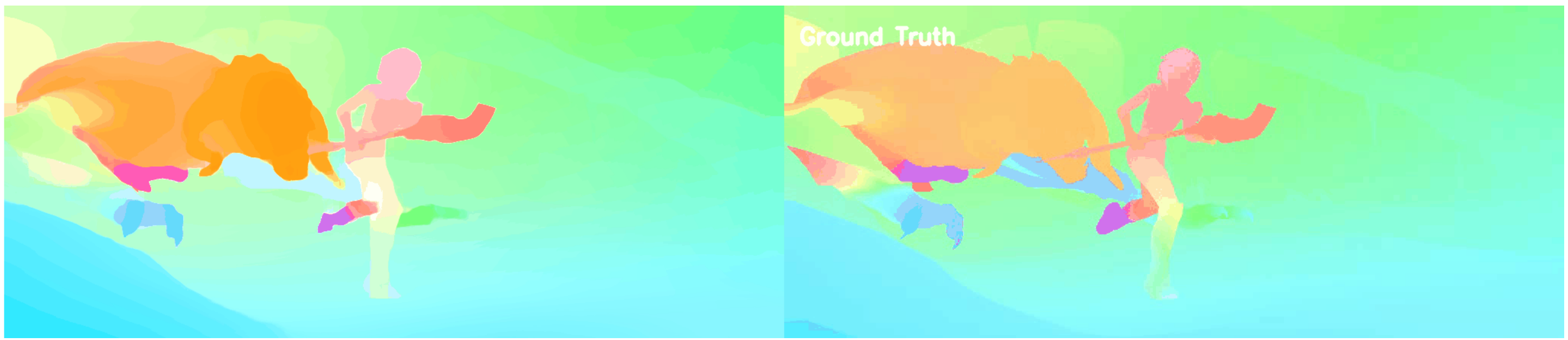
Early Fusion vs Late Fusion vs 3D CNN



Karpathy et al, "Large-scale Video Classification with Convolutional Neural Networks", CVPR 2014
Tran et al, "Learning Spatiotemporal Features with 3D Convolutional Networks", ICCV 2015

Recognizing Actions from Motion

We can easily recognize actions using only motion information

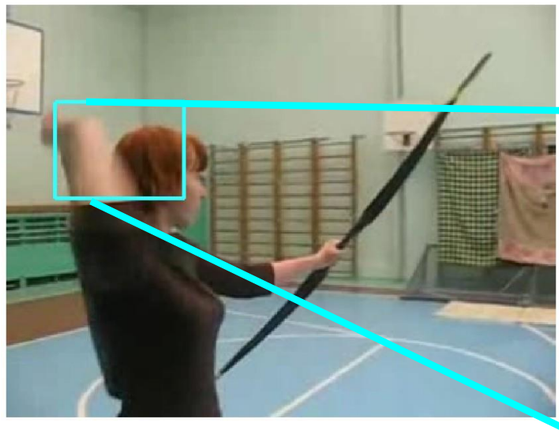


Johansson, “Visual perception of biological motion and a model for its analysis.” Perception & Psychophysics. 14(2):201-211. 1973.

Measuring Motion: Optical Flow

Optical Flow highlights local motion

Image at frame t



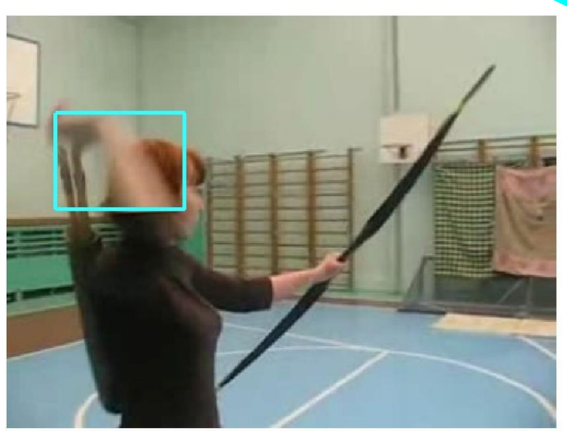
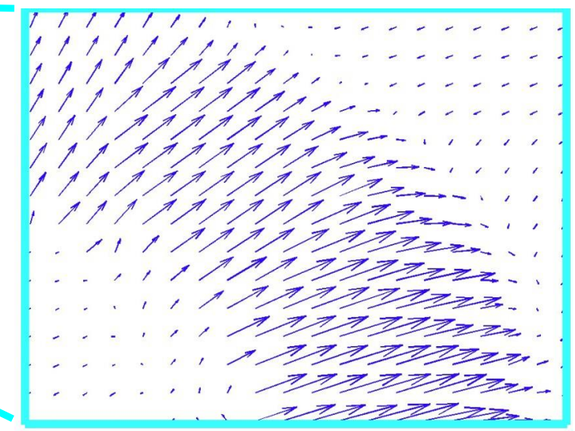


Image at frame t+1

Optical flow gives a displacement field F between images I_t and I_{t+1}

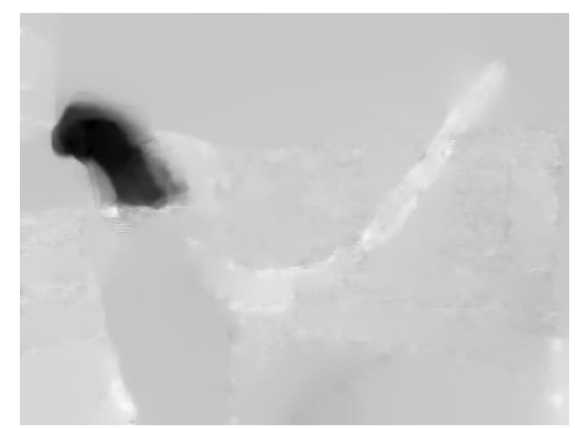


Tells where each pixel will move in the next frame:

$F(x, y) = (dx, dy)$

$I_{t+1}(x+dx, y+dy) = I_t(x, y)$

Horizontal flow dx

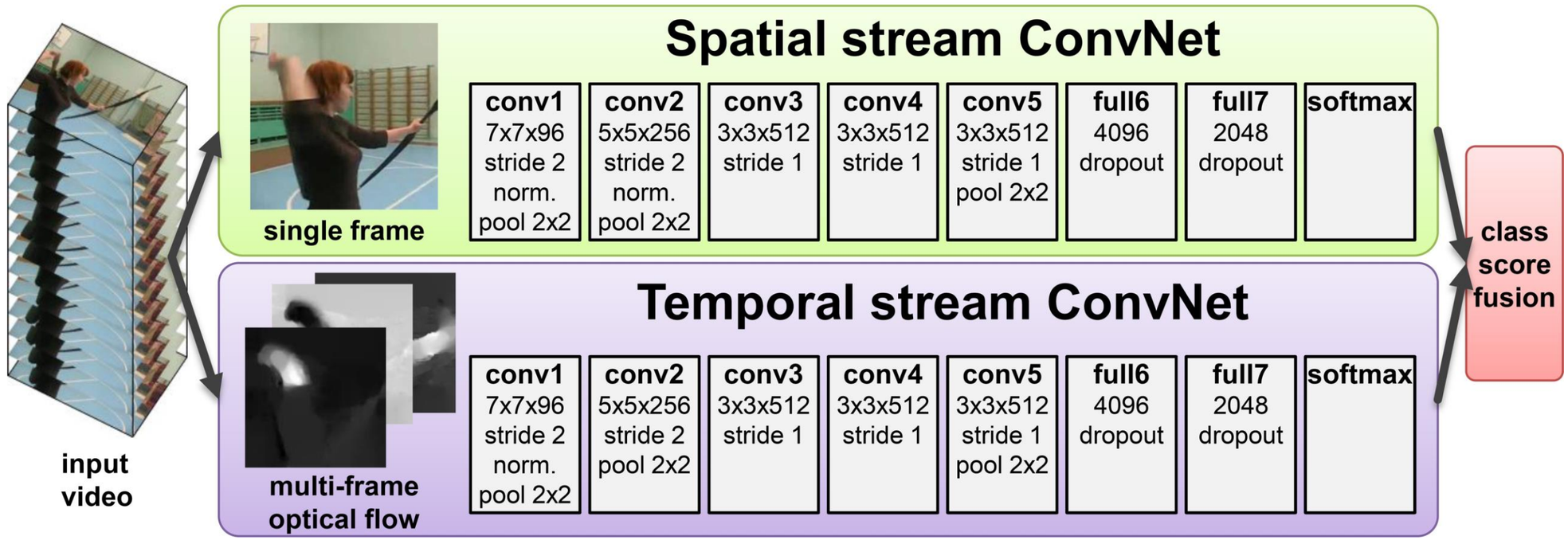


Vertical Flow dy

Simonyan and Zisserman, “Two-stream convolutional networks for action recognition in videos”, NeurIPS 2014

Separating Motion and Appearance: Two-Stream Networks

Input: Single Image
3 x H x W

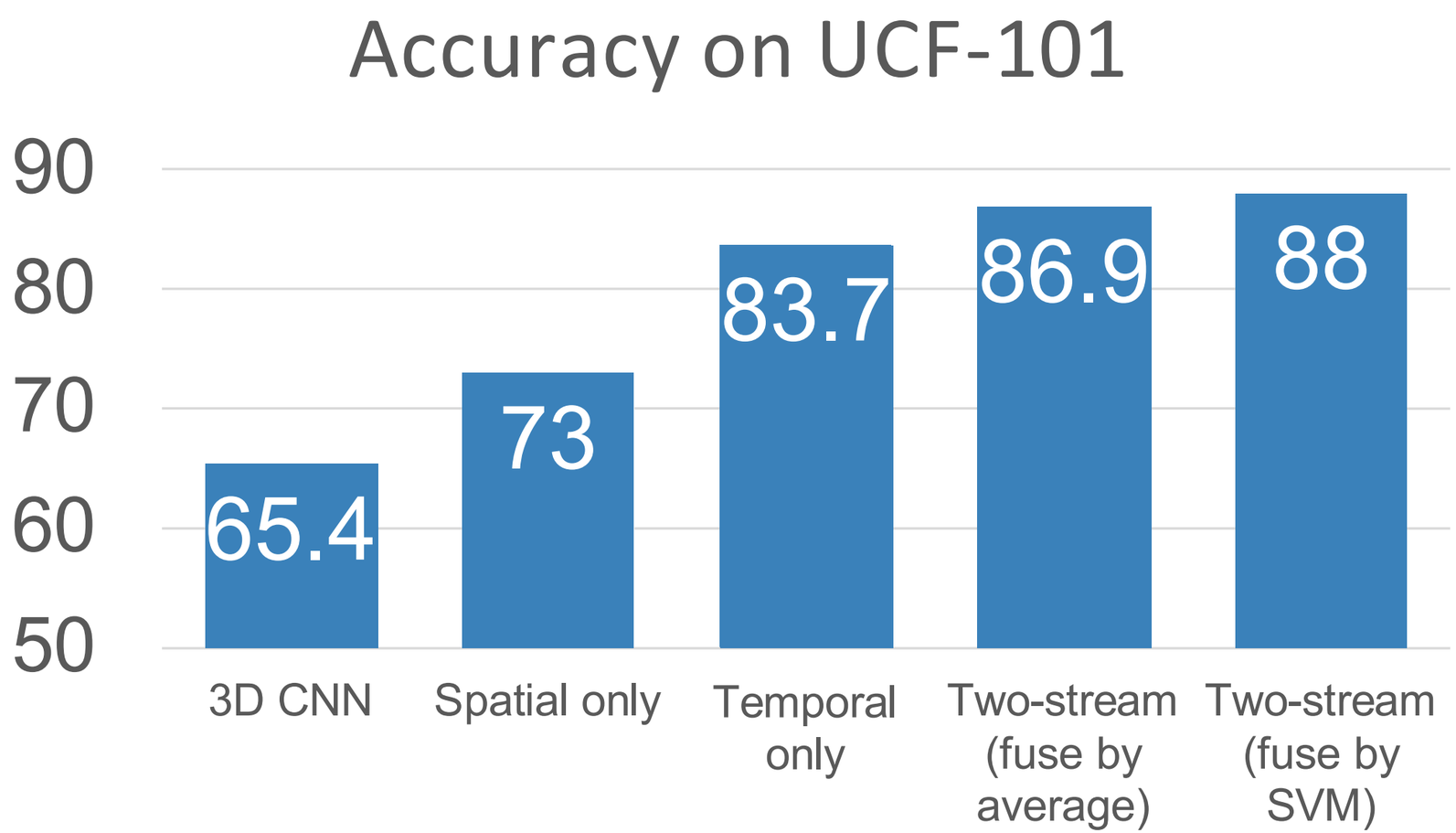


Input: Stack of optical flow:
[2*(T-1)] x H x W

Early fusion: First 2D conv
processes all flow images

Simonyan and Zisserman, "Two-stream convolutional networks for action recognition in videos", NeurIPS 2014

Separating Motion and Appearance: Two-Stream Networks

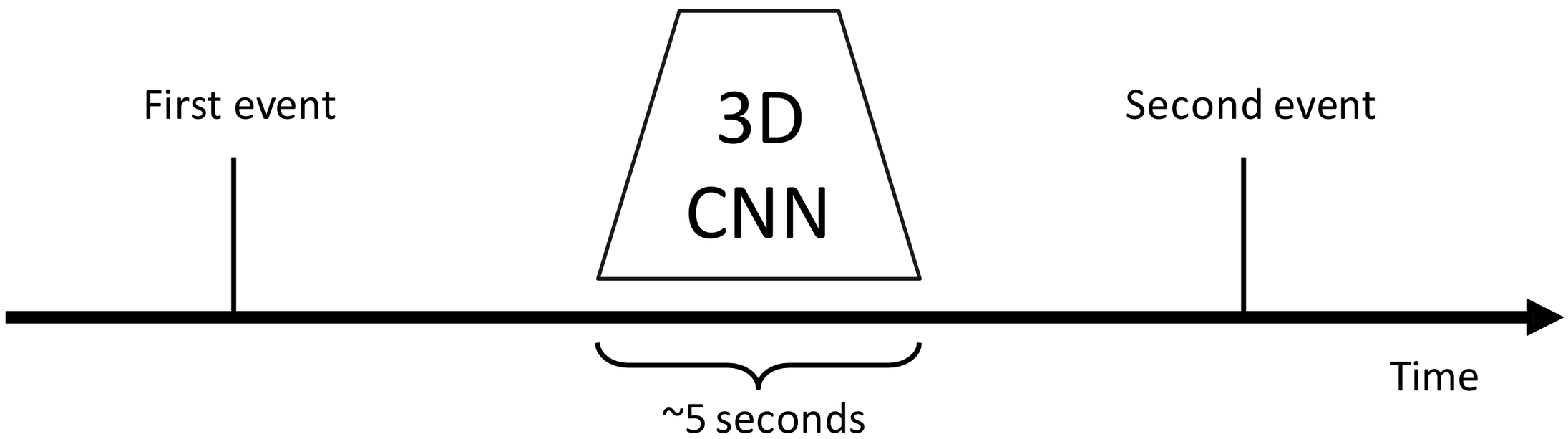


Simonyan and Zisserman, “Two-stream convolutional networks for action recognition in videos”, NeurIPS 2014

Modeling long-term temporal structure

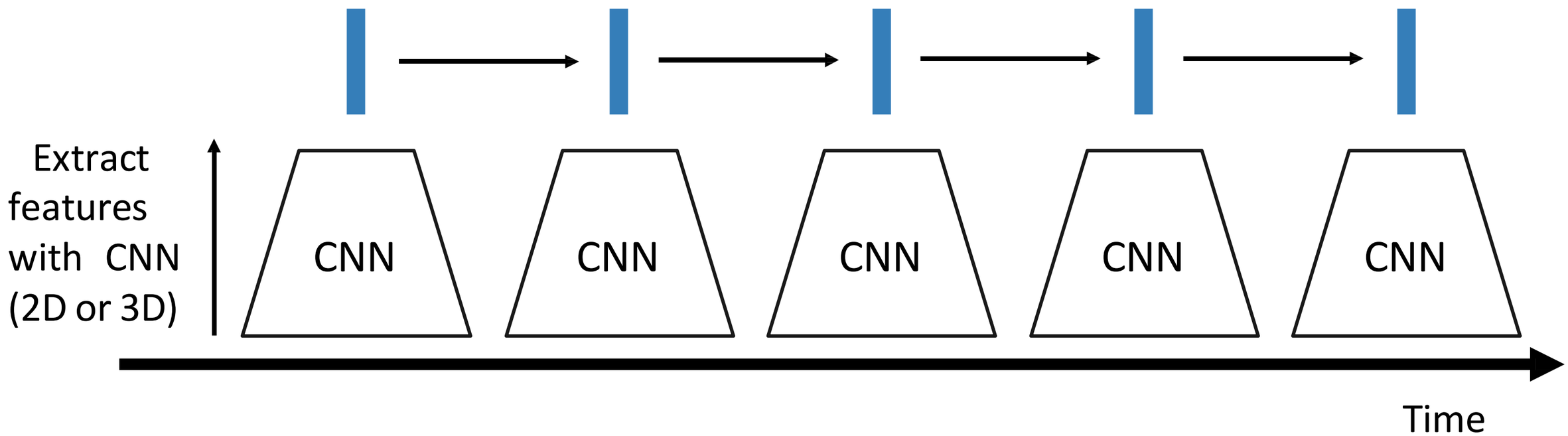
So far all our temporal CNNs only model local motion between frames in very short clips of ~2-5 seconds. What about long-term structure?

We know how to handle sequences! How about recurrent networks?



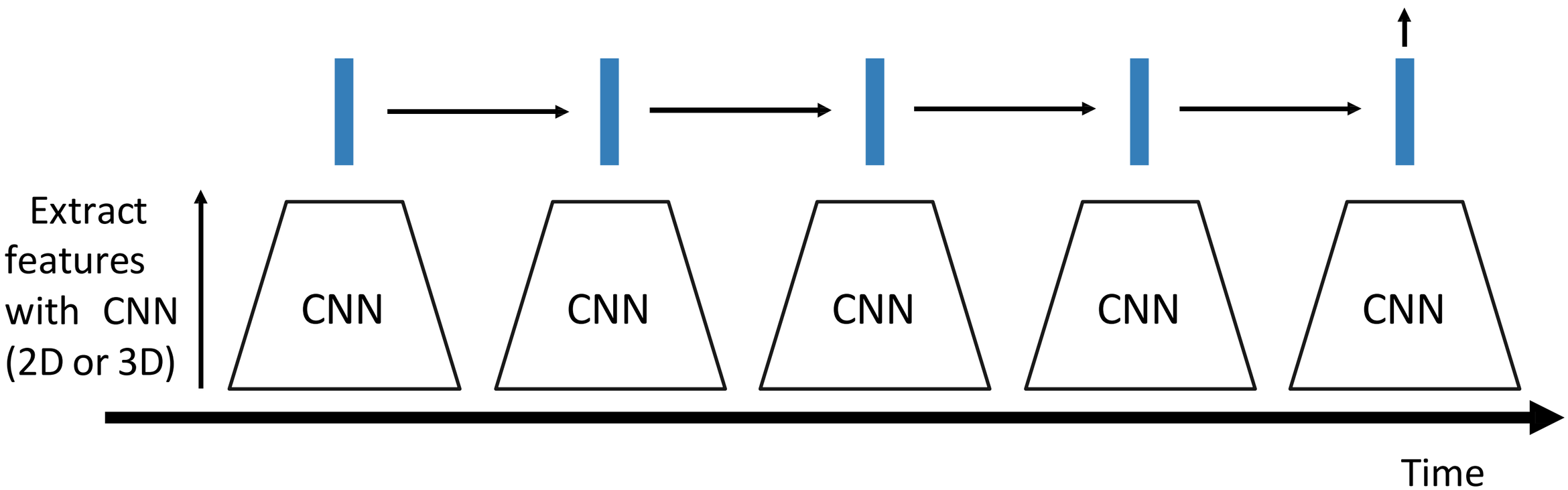
Modeling long-term temporal structure

Process local features using recurrent network (e.g. LSTM)



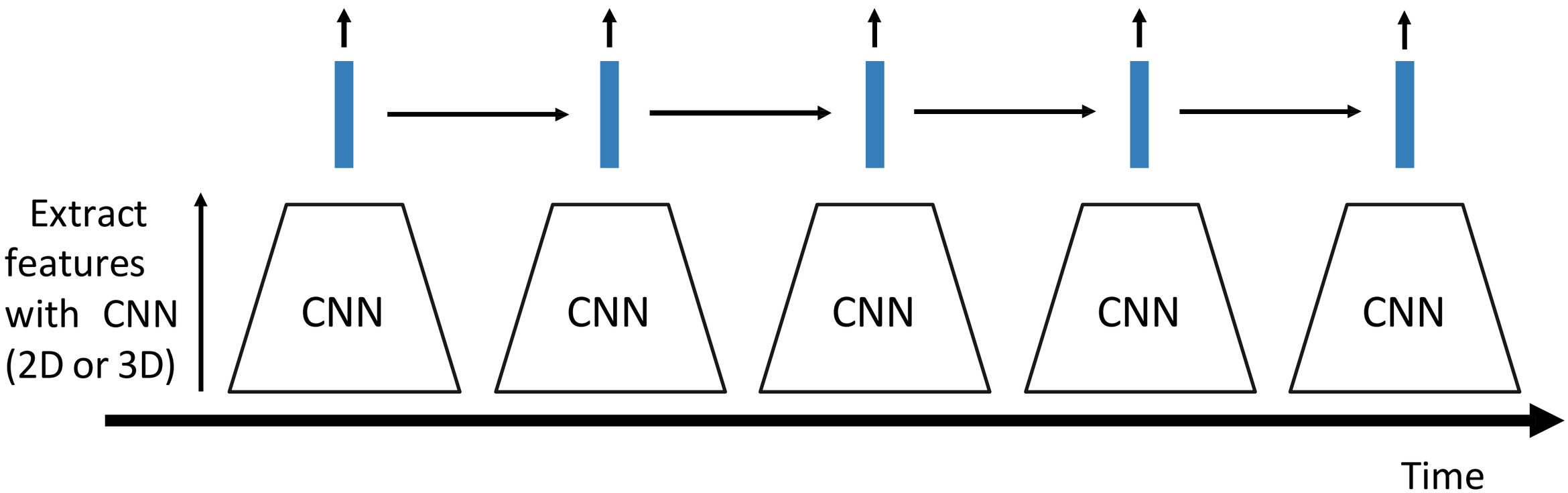
Modeling long-term temporal structure

Process local features using recurrent network (e.g. LSTM)
Many to one: One output at end of video



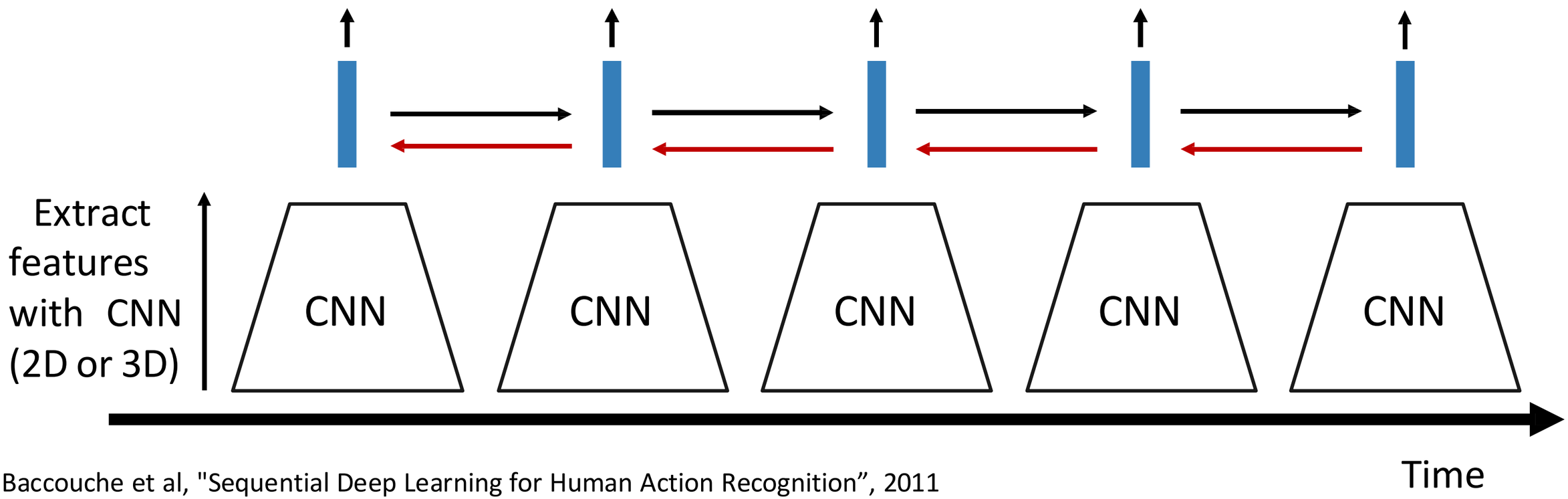
Modeling long-term temporal structure

Process local features using recurrent network (e.g. LSTM)
Many to many: one output per video frame



Modeling long-term temporal structure

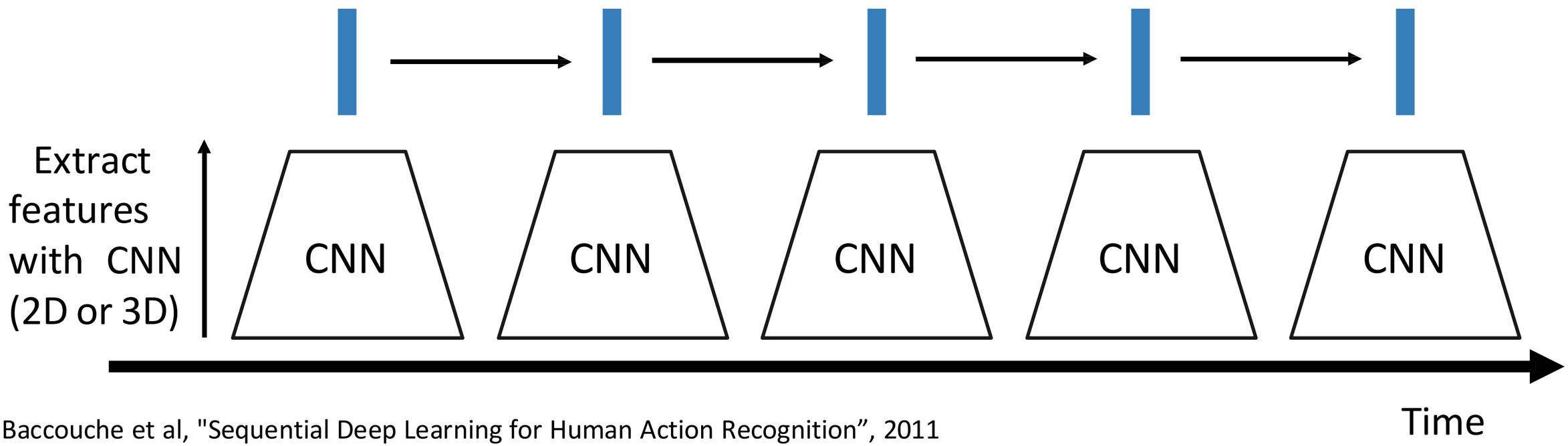
Sometimes don't backprop to CNN to save memory;
pretrain and use it as a feature extractor



Baccouche et al, "Sequential Deep Learning for Human Action Recognition", 2011
Donahue et al, "Long-term recurrent convolutional networks for visual recognition and description", CVPR 2015

Modeling long-term temporal structure

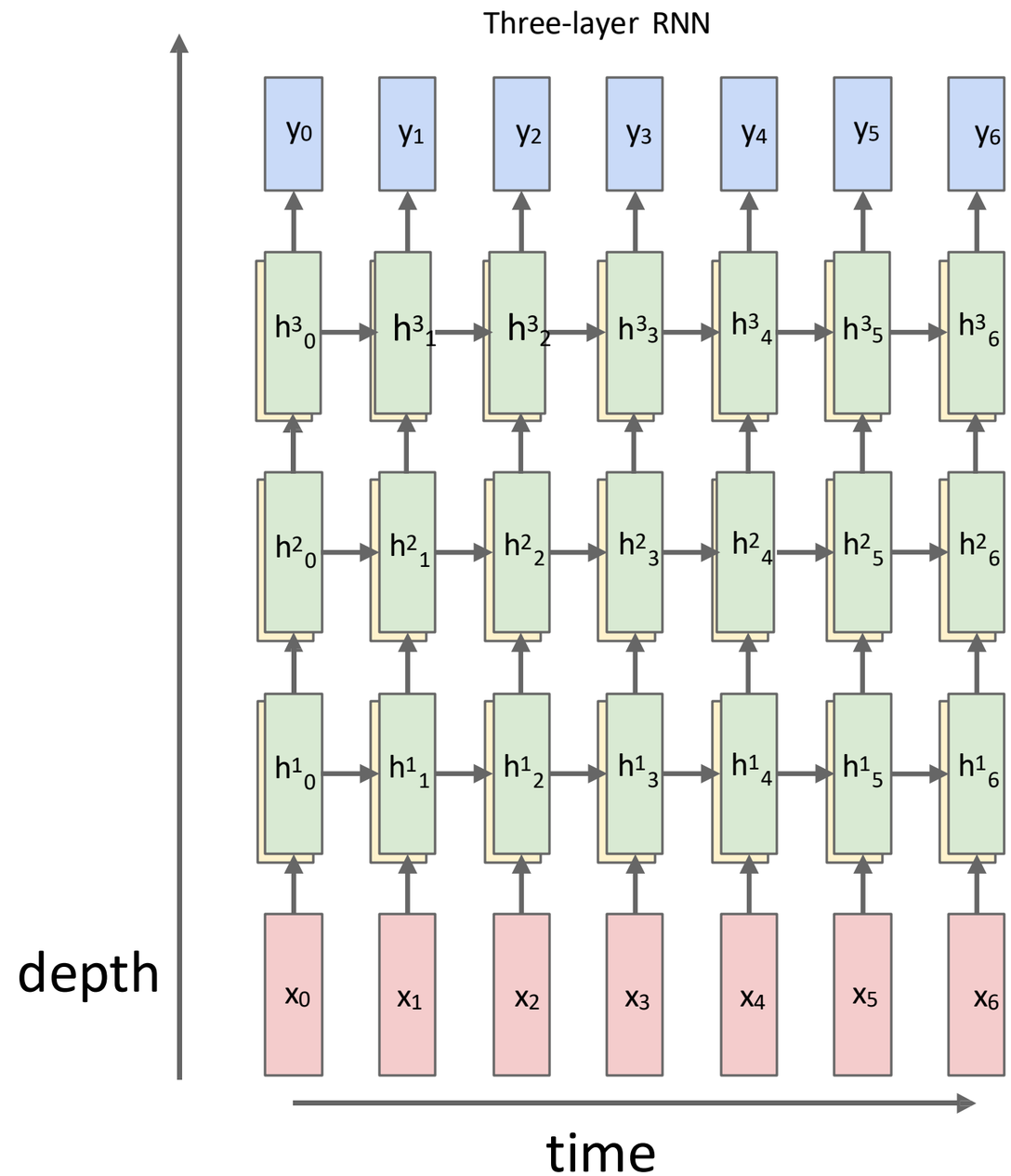
Inside CNN: Each value is a function of a fixed temporal window (local temporal structure)
Inside RNN: Each vector is a function of all previous vectors (global temporal structure)
Can we merge both approaches?



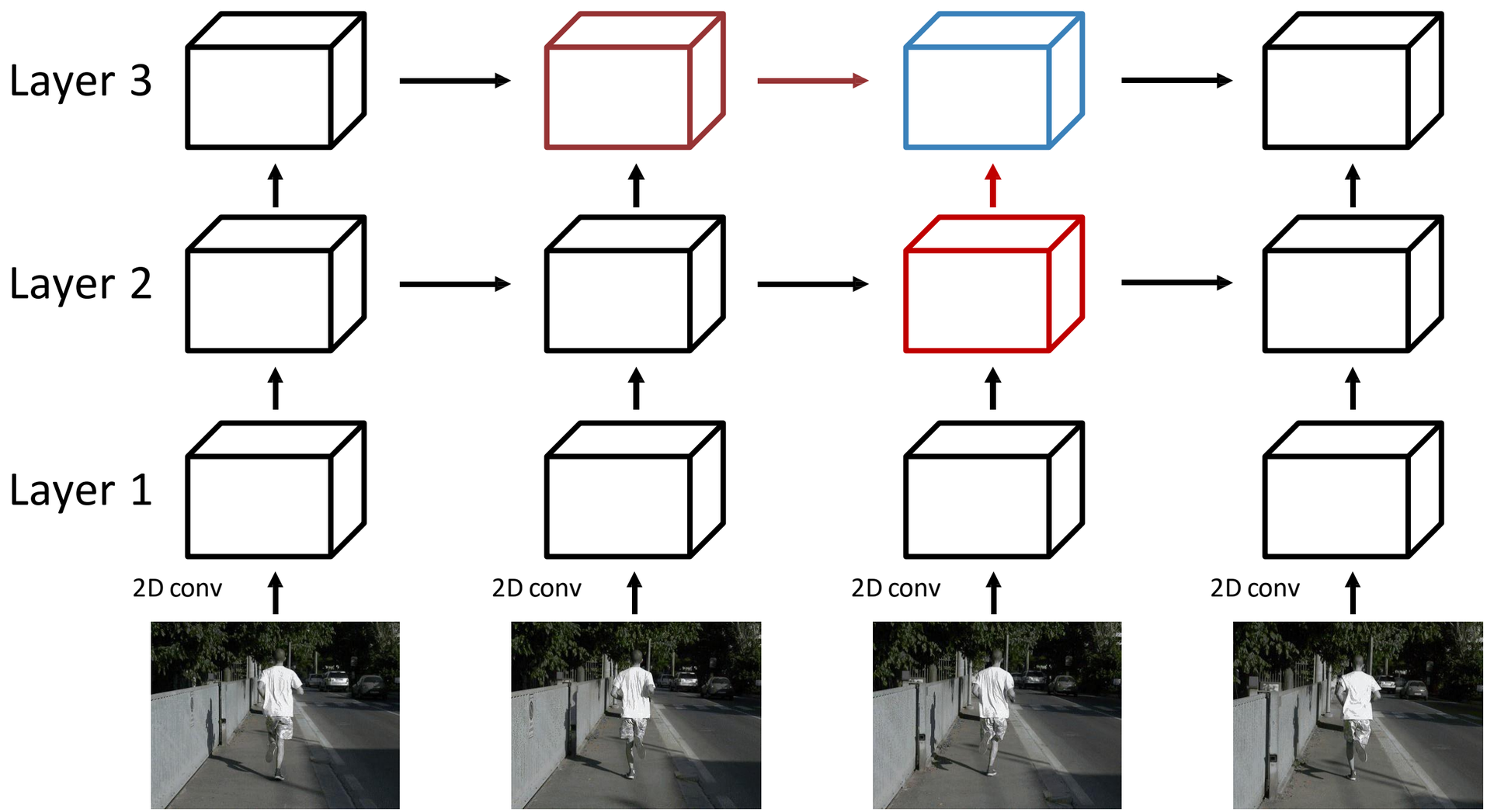
Baccouche et al, "Sequential Deep Learning for Human Action Recognition", 2011
Donahue et al, "Long-term recurrent convolutional networks for visual recognition and description", CVPR 2015

Recall: Multi-layer RNN

We can use a similar structure to process videos!



Recurrent Convolutional Network



Entire network uses 2D feature maps: C x H x W

Each depends on two inputs:
1. Same layer, previous timestep
2. Prev layer, same timestep

Use different weights at each layer, share weights across time

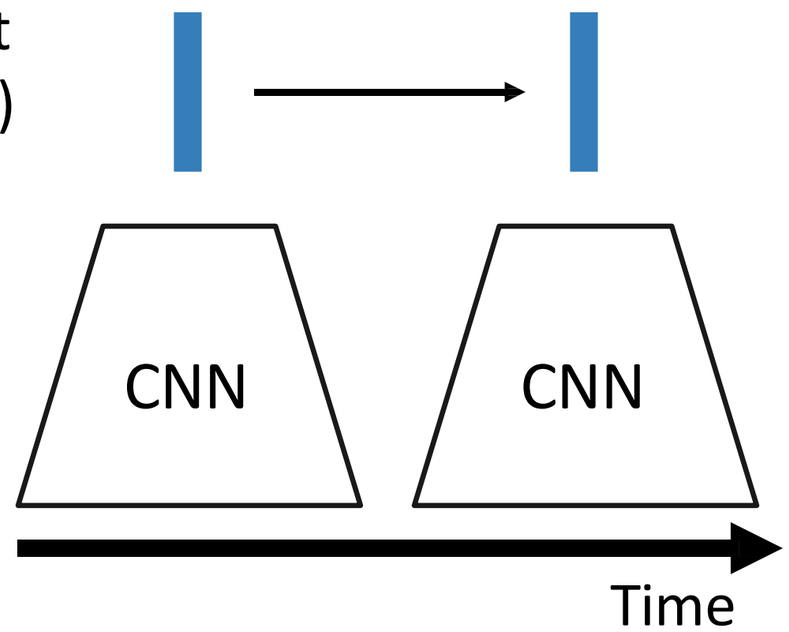
Ballas et al, "Delving Deeper into Convolutional Networks for Learning Video Representations", ICLR 2016

Modeling long-term temporal structure

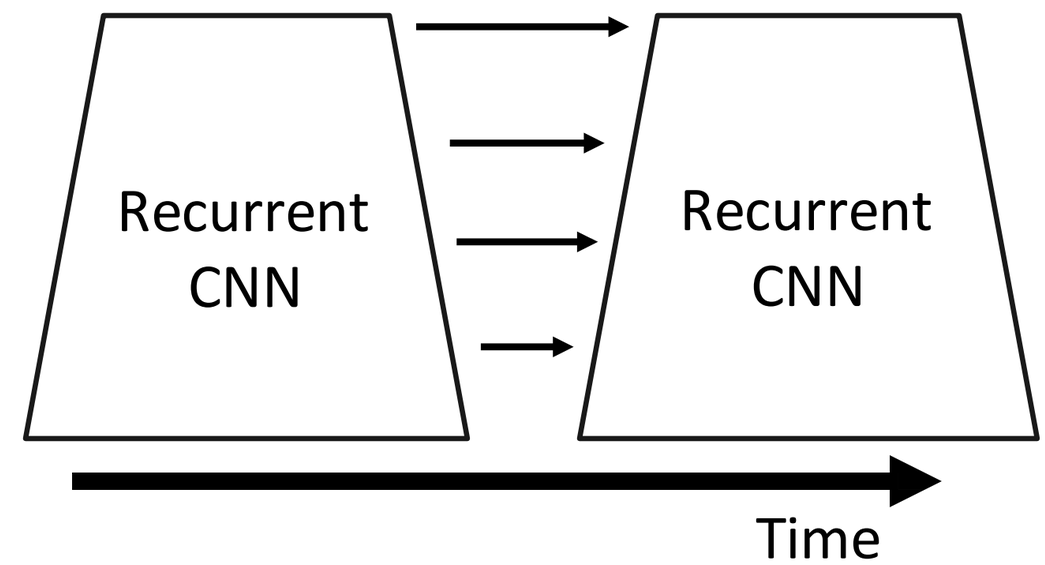
Problem: RNNs are slow for long sequences (can't be parallelized)

RNN: Infinite temporal extent (fully-connected)

CNN: finite temporal extent (convolutional)



Recurrent CNN: Infinite temporal extent (convolutional)

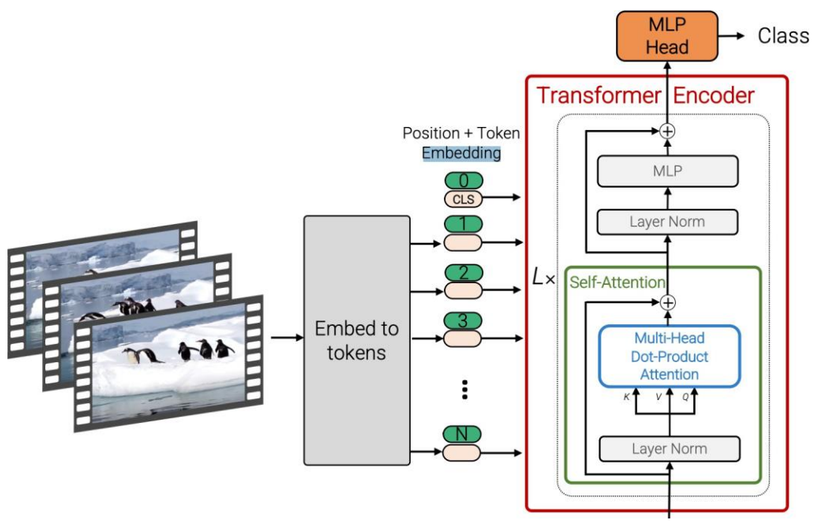


Baccouche et al, "Sequential Deep Learning for Human Action Recognition", 2011
Donahue et al, "Long-term recurrent convolutional networks for visual recognition and description", CVPR 2015

Ballas et al, "Delving Deeper into Convolutional Networks for Learning Video Representations", ICLR 2016

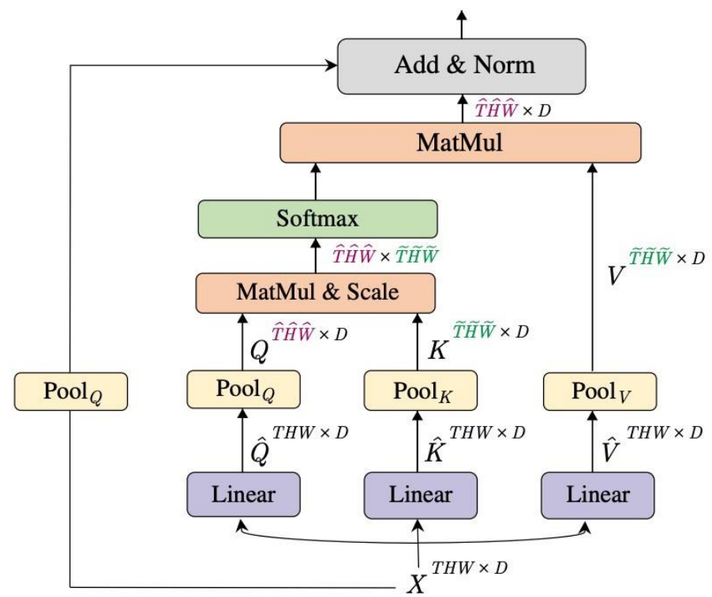
Vision Transformers for Video

Factorized attention:
Attend over space / time



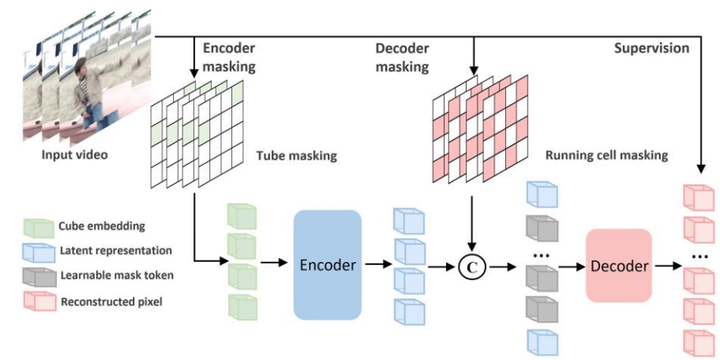
Bertasius et al, "Is Space-Time Attention All You Need for Video Understanding?", ICML 2021
Arnab et al, "ViViT: A Video Vision Transformer", ICCV 2021
Neimark et al, "Video Transformer Network", ICCV 2021

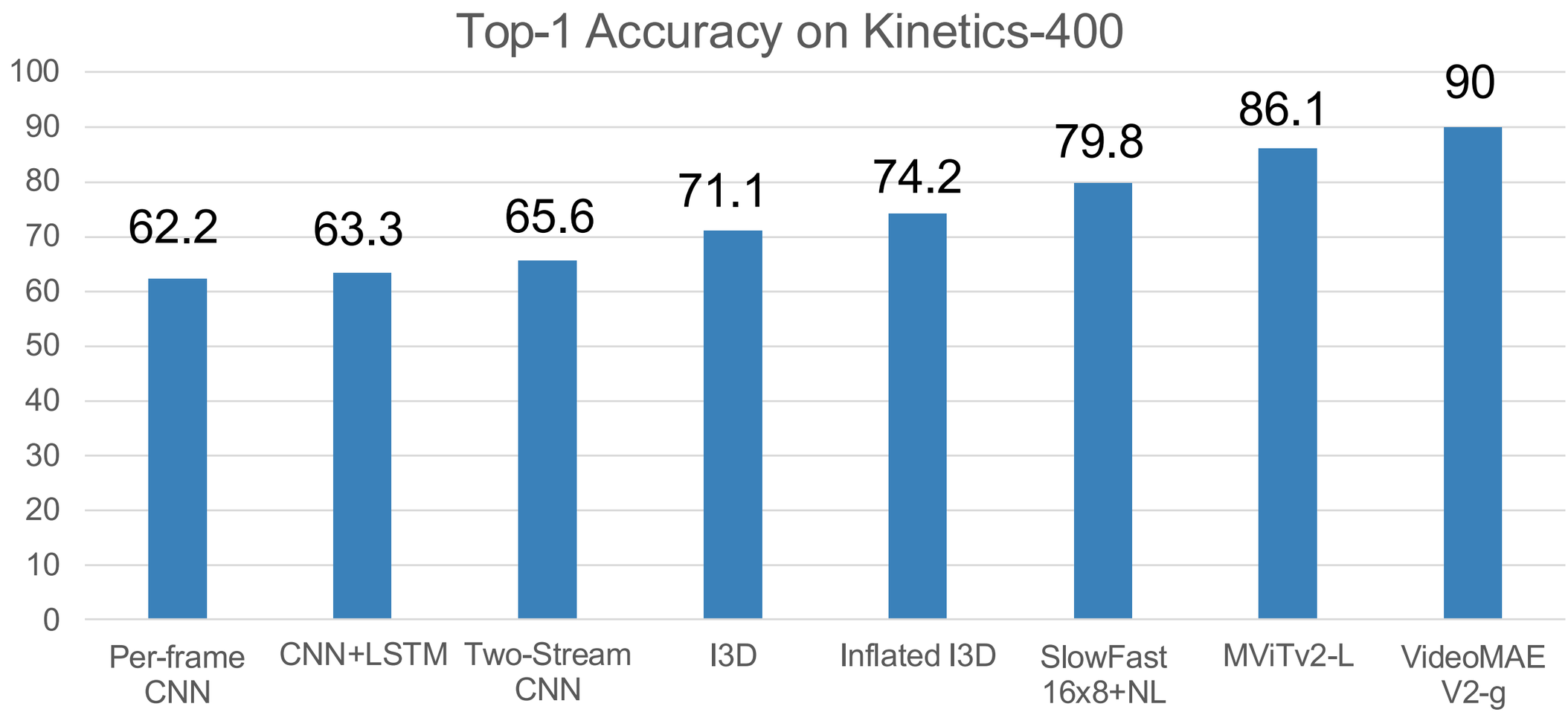
Pooling module:
Reduce number of tokens



Fan et al, "Multiscale Vision Transformers", ICCV 2021
Li et al, "MViTv2: Improved Multiscale Vision Transformers for Classification and Detection", CVPR 2022

Video masked autoencoders:
Efficient scalable pretraining



Wang et al. VideoMAE V2: Scaling Video Masked Autoencoders with Dual Making. CVPR 2023.
Tong et al. Video MAE: Masked Autoencoders are Data-Efficient Learners for Self-Supervised Video Pre-Training. NeurIPS 2022.
Feichtenhofer et al. Masked autoencoders as spatiotemporal learners. NeurIPS 2022.

Vision Transformers for Video